In [ ]:
```python
import marimo as mo
import time
from pysmt.shortcuts import (
    Symbol, BV, TRUE, And, Or, Not, Implies, Ite, Equals, NotEquals, Solver,
    BVAdd, BVSub, BVMul, BVAnd, BVLShl, BVLShr, BVExtract, BVZExt, BVSExt,
    BVULT, BVULE, BVSGE, simplify, substitute,
)
from pysmt.typing import BVType

# estados de controlo do CFA
LOOP, EVEN, ODD, STOP, ERROR = range(5)
NOME = {LOOP: "LOOP", EVEN: "EVEN", ODD: "ODD", STOP: "STOP", ERROR: "ERROR"}
```

In [ ]:
```python
class Mult:
    def __init__(self, n, com_sinal=False):
        self.n, self.com_sinal = n, com_sinal
        self.zero, self.um = BV(0, n), BV(1, n)

    def declare(self, i):
        # o pySMT guarda os símbolos por nome: a largura entra no nome para que
        # Mult(4) e Mult(8) (ou mudar o n no seletor) não colidam
        s = {'pc': Symbol(f'pc{i}', BVType(3))}
        for v in 'abxyz':
            s[v] = Symbol(f'{v}{i}_{self.n}', BVType(self.n))
        return s

    def esta(self, s, pc):
        return Equals(s['pc'], BV(pc, 3))

    def nao_neg(self, v):
        return BVSGE(v, self.zero) if self.com_sinal else TRUE()

    def init(self, s, extra=None):
        c = [self.esta(s, LOOP),
             Equals(s['x'], s['a']),
             Equals(s['y'], s['b']),
             Equals(s['z'], self.zero),
             self.nao_neg(s['a']),
             self.nao_neg(s['b'])]
        if extra is not None:
            c.append(extra(s))
        return And(c)

    # deteção explícita do overflow (os BitVec nunca o assinalam)

    def ovf_soma(self, s):
        # soma em n+1 bits; sem sinal o bit n é o carry
        n = self.n
        ext = BVSExt if self.com_sinal else BVZExt
        soma = BVAdd(ext(s['z'], 1), ext(s['x'], 1))
        if self.com_sinal:
            return NotEquals(BVExtract(soma, n, n), BVExtract(soma, n - 1, n - 1))
        return Equals(BVExtract(soma, n, n), BV(1, 1))
```

```
    def ovf_shift(self, s):
        x, n = s['x'], self.n
        if self.com_sinal:
            return NotEquals(BVExtract(x, n - 1, n - 1), BVExtract(x, n - 2, n -
2))
        return Equals(BVExtract(x, n - 1, n - 1), BV(1, 1))

    def guards(self, s):
        y = s['y']
        y0 = Equals(y, self.zero)
        par = Equals(BVAnd(y, self.um), self.zero)
        ovf_add, ovf_shl = self.ovf_soma(s), self.ovf_shift(s)
        return {
            'loop_stop': And(self.esta(s, LOOP), y0),
            'loop_even': And(self.esta(s, LOOP), Not(y0), par),
            'loop_odd':  And(self.esta(s, LOOP), Not(y0), Not(par)),
            'even_err':  And(self.esta(s, EVEN), ovf_shl),
            'even_loop': And(self.esta(s, EVEN), Not(ovf_shl)),
            'odd_err':   And(self.esta(s, ODD), ovf_add),
            'odd_loop':  And(self.esta(s, ODD), Not(ovf_add)),
            'stop':      self.esta(s, STOP),
            'err':       self.esta(s, ERROR),
        }

    def trans(self, s, p):
        g = self.guards(s)

        def upd(pc, **novos):
            # pc' = pc; x, y, z mudam para `novos[v]` ou mantêm-se
            return And(self.esta(p, pc),
                       *[Equals(p[v], novos.get(v, s[v])) for v in 'xyz'])

        return And(
            Equals(p['a'], s['a']),
            Equals(p['b'], s['b']),
            Or(
                And(g['loop_stop'], upd(STOP)),
                And(g['loop_even'], upd(EVEN)),
                And(g['loop_odd'],  upd(ODD)),
                And(g['even_err'],  upd(ERROR)),
                And(g['even_loop'], upd(LOOP, x=BVLShl(s['x'], self.um),
                                              y=BVLShr(s['y'], self.um))),
                And(g['odd_err'],   upd(ERROR)),
                And(g['odd_loop'],  upd(LOOP, y=BVSub(s['y'], self.um),
                                              z=BVAdd(s['z'], s['x']))),
                And(g['stop'],      upd(STOP)),
                And(g['err'],       upd(ERROR)),
            ),
        )

    def erro(self, s):
        return self.esta(s, ERROR)

    def terminal(self, s):
        return Or(self.esta(s, STOP), self.esta(s, ERROR))
```

```
    def W(self, v):
        # extensão para 2n bits: aritmética exata
        return BVSExt(v, self.n) if self.com_sinal else BVZExt(v, self.n)
```

In [ ]:
```
def resolve(*formulas):
    """Modelo das fórmulas, ou None se forem insatisfazíveis."""
    # O solver Z3 recebe uma coleção de restrições lógicas e tenta satisfazer
todas ao mesmo tempo.
    # Se a conjunção das fórmulas for satisfazível, devolve um modelo concreto
para as variáveis;
    # caso contrário, devolve None, que interpretamos como "fórmulas
inconsistentes".
    with Solver(name="z3") as solver:
        for f in formulas:
            solver.add_assertion(f)
        return solver.get_model() if solver.solve() else None
```

In [ ]:
```
def unroll(M, k):
    # A relação de transição T(s,s') descreve um passo. Para um caminho de k
passos, precisamos de
    # k+1 instâncias de estado: s0, s1, ..., sk. Cada estado é um dicionário com
pc, a, b, x, y, z.
    # O resultado devolve os estados simbólicos e as restrições de transição entre
estados consecutivos.
    tr = [M.declare(i) for i in range(k + 1)]
    return tr, [M.trans(tr[i], tr[i + 1]) for i in range(k)]
```

In [ ]:
```
def valor(M, m, e):
    # Converte um valor de BitVec resultado do solver para um inteiro Python.
    # Em modo com sinal, os BitVec têm representação em complemento para dois;
    # quando o bit mais significativo está a 1, devemos interpretar o valor como
negativo.
    v = m.get_py_value(e)
    if M.com_sinal and v >= 2 ** (M.n - 1):
        v -= 2 ** M.n
    return v
```

In [ ]:
```
def mostra(M, m, tr):
    # Esta rotina converte um traço simbólico obtido do solver numa tabela legível
para o humano.
    # Cada estado do traço corresponde a um instante de execução da máquina e
contém valores concretos
    # para pc, x, y e z. O objetivo é mostrar a sequência de estados até atingir
STOP ou ERROR.
    linhas = [f"{'t':>3} {'pc':>6} {'x':>6} {'y':>6} {'z':>6}   x*y+z"]
    for i, s in enumerate(tr):
        pc = m.get_py_value(s['pc'])
        x, y, z = (valor(M, m, s[v]) for v in 'xyz')
        linhas.append(f"{i:>3} {NOME[pc]:>6} {x:>6} {y:>6} {z:>6}   {x * y + z}")
        if pc in (STOP, ERROR):
            break
    return "\n".join(linhas)
```

In [ ]:
```
def kinduction_always(M, P, k, extra=None):
    # Formato da ficha 8 para um k fixo (declare/init/trans vêm de M):
```

```
    #   caso base: k estados s0..s(k-1) com init(s0), trans entre consecutivos e
algum P(si) falso;
    #   passo:     k+1 estados com trans entre consecutivos, P nos k primeiros e P
falso no último.
    # Devolve None se as duas verificações são insatisfazíveis, ou
("base"|"passo", (modelo, estados)).
    tr, T = unroll(M, k - 1)
    m = resolve(M.init(tr[0], extra), *T, Or([Not(P(s)) for s in tr]))
    if m is not None:
        return "base", (m, tr)
    tr, T = unroll(M, k)
    m = resolve(*T, *[P(s) for s in tr[:-1]], Not(P(tr[-1])))
    if m is not None:
        return "passo", (m, tr)
    return None
```

In [ ]:
```
def kinducao(M, P, K, extra=None):
    # Tenta k = 1, 2, ..., K com `kinduction_always` e pára no primeiro k que
prova P.
    # Se a base falhar, P é falso (contra-exemplo real); se só o passo falhar,
tenta-se k maior.
    for k in range(1, K + 1):
        falha = kinduction_always(M, P, k, extra)
        if falha is None:
            return f"PROVADO com k={k}", None
        if falha[0] == "base":
            return f"FALSO (contra-exemplo no caso base, k={k})", falha[1]
    return f"INCONCLUSIVO até k={K}", None
```

In [ ]:
```
def invariantes(M):
    # Invariante do programa em aritmética modular (BitVec) e versão exata em 2n
bits.
    # A primeira versão afirma que x*y + z = a*b, mas sem contexto extra pode
falhar em estados não alcançáveis.
    # A segunda versão reforça a invariantização com extensão para 2n bits e exige
x,y,z >= 0, eliminando estados
    # inconsistentes para o caso com sinal.
    W = M.W
    P_mod = lambda s: Equals(BVAdd(BVMul(s['x'], s['y']), s['z']),
                             BVMul(s['a'], s['b']))
    P_exato = lambda s: And(
        Equals(BVAdd(BVMul(W(s['x']), W(s['y'])), W(s['z'])),
               BVMul(W(s['a']), W(s['b']))),
        M.nao_neg(s['x']), M.nao_neg(s['y']), M.nao_neg(s['z']))   # >= 0 só conta
com sinal
    return P_mod, P_exato
```

In [ ]:
```
def bmc(M, mau, K, extra=None):
    """Primeiro k <= K em que `mau` é alcançável: (k, modelo, estados)."""
    # BMC (Bounded Model Checking) cria um caminho progressivo de estados desde o
início do programa e,
    # para cada comprimento k, pergunta se existe alguma execução de tamanho k que
viola a propriedade mau.
    # Quando encontra um modelo, devolve o passo em que a propriedade falha e o
traço correspondente.
```

```
    with Solver(name="z3") as solver:
        tr = [M.declare(0)]
        solver.add_assertion(M.init(tr[0], extra))
        for k in range(K + 1):
            if k > 0:
                tr.append(M.declare(k))
                solver.add_assertion(M.trans(tr[k - 1], tr[k]))
            solver.push()
            solver.add_assertion(mau(tr[k]))
            if solver.solve():
                return k, solver.get_model(), tr
            solver.pop()
    return None
```

In [ ]:
```
def termina_J(M):
    """Invariante auxiliar J: pc <= 4, ODD => y ímpar, EVEN => y par e != 0."""
    # J estrutura a informação semântica que usaremos para provar terminação: o
programa só pode estar num
    # dos nós do CFA e, em cada uma das ramificações ODD/EVEN, y tem a paridade
correta.
    impar = lambda s: Equals(BVAnd(s['y'], M.um), M.um)
    par = lambda s: Equals(BVAnd(s['y'], M.um), M.zero)
    return lambda s: And(BVULE(s['pc'], BV(4, 3)),
                         Implies(M.esta(s, ODD), impar(s)),
                         Implies(M.esta(s, EVEN), And(par(s), NotEquals(s['y'],
M.zero))))
```

In [ ]:
```
def termina_V(M):
    """Variante V = 3y + r(pc), calculada em n+2 bits sem sinal (não há
overflow)."""
    # V é uma medida de progressão: em LOOP vale mais do que em EVEN/ODD, e estes
valem mais do que STOP/ERROR.
    # Como y diminui ao longo das iterações, a medida garante que a execução não
pode ciclar indefinidamente.
    w = M.n + 2
    r_pc = lambda s: Ite(M.esta(s, LOOP), BV(2, w),
                         Ite(Or(M.esta(s, ODD), M.esta(s, EVEN)), BV(1, w), BV(0,
w)))
    return lambda s: BVAdd(BVMul(BV(3, w), BVZExt(s['y'], 2)), r_pc(s))
```

In [ ]:
```
def decresce(M, J, V):
    """True se J(s) /\\ ~terminal(s) /\\ T(s,s') => V(s') < V(s) (fórmula
insatisfazível)."""
    # Esta checagem formaliza a segunda condição da prova de terminação: a partir
de qualquer estado
    # que respeita J, não terminal, e que tenha um passo válido da transição, a
variante deve diminuir.
    # Se a conjunção for satisfazível, então existe um contra-exemplo em que o
sistema avança sem progredir,
    # o que invalida a garantia de terminação.
    s0, s1 = M.declare(0), M.declare(1)
    return resolve(J(s0), Not(M.terminal(s0)), M.trans(s0, s1),
                   Not(BVULT(V(s1), V(s0)))) is None
```

In [ ]:
```python
def sem_bloqueio(M, J):
    """True se todo o estado não terminal que satisfaz J tem alguma guarda ativa."""
    # Um sistema pode ser "bem-fundado" em termos de variante, mas ainda assim bloquear-se se não houver
    # transição ativa a partir de um estado não terminal. A verificação de ausência de bloqueio garante que
    # cada estado compatível com J tem pelo menos uma guarda que se pode disparar.
    s0 = M.declare(0)
    return resolve(J(s0), Not(M.terminal(s0)),
                   Not(Or(list(M.guards(s0).values())))) is None
```

TP2 — Exercício 3: multiplicação de inteiros como SFOTS (BitVec, pySMT)

Raciocínio do trabalho. O programa (multiplicação por somas e deslocamentos) é modelado como um sistema de transições (SFOTS) sobre `BitVec`, o que reproduz o comportamento real do hardware, incluindo o *wrap-around*. Como os `BitVec` não assinalam overflow, este é detetado explicitamente e leva a um estado `ERROR`. Sobre este modelo fazem-se quatro perguntas, cada uma com a técnica adequada:

1. **Execução** — *unrolling* da relação de transição para ver um traço concreto;
2. **Invariante** $x \cdot y + z = a \cdot b$ — k-indução (prova) e BMC (procura de contra-exemplos);
3. **Segurança** (G $\neg \mathrm{ERROR}$ sob $a \cdot b < 2^n$) — k-indução e prova analítica;
4. **Terminação** — variante V e invariante auxiliar J.

O solver valida o modelo para $n = 8$. Como o custo da k-indução cresce muito com a largura (multiplicações de n e $2n$ bits), a generalização a qualquer n é feita por **prova analítica** (secção 6), que se apoia no mesmo invariante e na mesma variante.

```
INPUT a, b                          # n bits, a, b >= 0
x, y, z = a, b, 0
LOOP:  while y != 0:
ODD:       if y & 1 == 1:  x, y, z = x, y-1, z+x      (overflow em
z+x  → ERROR)
EVEN:      else:           x, y, z = x<<1, y>>1, z    (overflow em
x<<1 → ERROR)
STOP:  assert z == a*b
```

In [ ]:
```python
# Seleção do modo de aritmética (n é fixo a 8): em modo com sinal as comparações de grandeza
# e as extensões passam a usar semântica de complemento para dois; sem sinal, o modelo é mais simples.
sinal_ui = mo.ui.switch(label="com sinal")
K_MAX = 6     # limite para a k-indução (invariante e segurança reforçada)
sinal_ui
```

com sinal

1. O SFOTS (classe `Mult`)

- **Variáveis:** `pc` é um `BitVec(3)`; `a, b, x, y, z` são `BitVec(n)`. `a` e `b` não mudam.
- **Estado inicial:** $pc = \mathrm{LOOP} \wedge x = a \wedge y = b \wedge z = 0 \wedge a \geq 0 \wedge b \geq 0$ (a condição ≥ 0 só tem efeito no modo com sinal).
- **Transição $T(s, s')$:** disjunção das ligações do CFA, cada uma com a sua guarda e a atualização das variáveis (as não mencionadas mantêm-se). As guardas normais de `EVEN` e `ODD` excluem o caso de overflow, o que dá a precedência ao erro.
- **Erro:** $pc = \mathrm{ERROR}$.

In [ ]:
```
# Instancia o modelo de verificação com n = 8 e o sinal escolhido. Todas as
funções que seguem
# usam a mesma instância `M`, para que as provas e os contra-exemplos correspondam
ao mesmo sistema.
# (n = 16 foi retirado: a k-indução com multiplicações de 16 bits demora
demasiado.)
n = 8
M = Mult(n, sinal_ui.value)
```

Execução de exemplo

Uma execução concreta obtida por *unrolling* da relação de transição.

In [ ]:
```
# Escolhe valores iniciais e pergunta ao solver se existe algum caminho válido do
estado inicial até
# o final para esses valores. Quando existe, mostramos o traço concreto como
depuração do sistema.
_a0, _b0 = (11, 17) if n >= 5 else (5, 3)
_tr, _T = unroll(M, 4 * n)
_m = resolve(M.init(_tr[0]), Equals(_tr[0]['a'], BV(_a0, n)),
             Equals(_tr[0]['b'], BV(_b0, n)), *_T)
_txt = ("sem traço" if _m is None
        else f"a={_a0}, b={_b0}, a*b={_a0 * _b0}\n\n" + mostra(M, _m, _tr))
mo.md(f"```\n{_txt}\n```")
```

```
a=11, b=17, a*b=187

t pc x y z x*y+z 0 LOOP 11 17 0 187 1 ODD 11 17 0 187 2 LOOP 11 16 11 187 3 EVEN 11 16 11
187 4 LOOP 22 8 11 187 5 EVEN 22 8 11 187 6 LOOP 44 4 11 187 7 EVEN 44 4 11 187 8 LOOP
88 2 11 187 9 EVEN 88 2 11 187 10 LOOP 176 1 11 187 11 ODD 176 1 11 187 12 LOOP 176 0
187 187 13 STOP 176 0 187 187
```

2. O invariante $x \cdot y + z = a \cdot b$ por k-indução

- **caso base:** nenhum traço com k estados a partir de I viola P;
- **passo:** $P(s_0) \wedge \cdots \wedge P(s_{k-1}) \wedge T(s_0, s_1) \wedge \cdots \wedge T(s_{k-1}, s_k) \Rightarrow P(s_k)$.

Verifica-se em duas versões: **módulo** 2^n (aritmética `BitVec` usual) e **exata**, em $2n$ bits (`BVZExt`, ou `BVSExt` com sinal, e $x, y, z \geq 0$ para excluir estados inalcançáveis com $y < 0$). Com $k = 1$ o passo falha: estados inalcançáveis como $pc = \mathrm{EVEN}$ com y ímpar satisfazem P mas o sucessor não. Com $k = 2$ o predecessor obriga a y par.

In [ ]:
```
# Testa a propriedade do invariante em duas formulações: a modular e a exata.
# A versão modular é a mais direta, mas pode admitir estados impossíveis que não são alcançáveis;
# a versão exata reforça a propriedade com extensões de largura e condições de não negatividade.
P_mod, P_exato = invariantes(M)
_t0 = time.time()
_r_mod = kinducao(M, P_mod, K_MAX)[0]
_r_ex = kinducao(M, P_exato, K_MAX)[0]
mo.md(f"""
* invariante módulo $2^n$: **{_r_mod}**
* invariante exato (2n bits): **{_r_ex}**

({time.time() - _t0:.1f}s). Em `STOP` tem-se $y=0$, logo $z=a\\cdot b$: a asserção final é válida sempre que se chega a `STOP`.
""")
```

invariante módulo 2^n: **PROVADO com k=2**
invariante exato (2n bits): **PROVADO com k=2**
(100.9s). Em `STOP` tem-se $y = 0$, logo $z = a \cdot b$: a asserção final é válida sempre que se chega a `STOP`.

3. Segurança com $a < 2^{n/2} \wedge b < 2^{n/2}$

Primeiro mostra-se, por BMC, que **sem** a condição o erro é alcançável. Depois prova-se $\mathsf{G}\,\neg\mathrm{ERROR}$ **com** a condição, de duas formas:

- **diretamente** com $P \equiv \neg\mathrm{ERROR} \wedge a, b < 2^{n/2}$ (como a, b são constantes, a condição inclui-se em P);
- com a propriedade **reforçada** $P^+ \equiv \neg\mathrm{ERROR} \wedge a, b < 2^{n/2} \wedge xy + z = ab$: em `ODD`, $y \geq 1$ e $x + z \leq xy + z = ab < 2^n$ (sem *carry*); em `EVEN`, $y \geq 2$ e $2x \leq xy \leq ab < 2^n$.

In [ ]:
```
_r = bmc(M, M.erro, 4 * n)
if _r is None:
    _txt = f"ERROR não alcançável até {4 * n} transições"
else:
    _k, _m, _tr = _r
    _txt = (f"SEM restrição: ERROR alcançável em {_k} transições, "
```

```
            f"com a={valor(M, _m, _tr[0]['a'])}, b={valor(M, _m, _tr[0]
['b'])}\n\n"
            + mostra(M, _m, _tr))
mo.md(f"```\n{_txt}\n```")
```

```
SEM restrição: ERROR alcançável em 2 transições, com a=129, b=128
```

t pc x y z x*y+z 0 LOOP 129 128 0 16512 1 EVEN 129 128 0 16512 2 ERROR 129 128 0 16512

In [ ]:
```
_h = BV(2 ** (n // 2), n)
pequenos = lambda s: And(BVULT(s['a'], _h), BVULT(s['b'], _h))
seguro = lambda s: And(Not(M.erro(s)), pequenos(s))
seguro_r = lambda s: And(seguro(s), P_exato(s))

_t0 = time.time()
_r_ref = kinducao(M, seguro_r, K_MAX, extra=pequenos)[0]
_t1 = time.time() - _t0
_t0 = time.time()
_r_dir = kinducao(M, seguro, 4 * n + 2, extra=pequenos)[0]
_t2 = time.time() - _t0
mo.md(f"""
* reforçada com o invariante exato: **{_r_ref}** ({_t1:.1f}s)
* direta (sem reforço): **{_r_dir}** ({_t2:.1f}s)

A prova direta só tem sucesso quando $k$ atinge o comprimento do caminho mais
longo, perto de $4n$.
Com o reforço por $xy+z=ab$ basta $k=2$.
""")
```

reforçada com o invariante exato: **PROVADO com k=2** (1.6s)
direta (sem reforço): **PROVADO com k=32** (8.5s)
A prova direta só tem sucesso quando k atinge o comprimento do caminho mais longo, perto de $4n$. Com o reforço por $xy + z = ab$ basta $k = 2$.

4. Terminação (propriedade de animação)

Para quaisquer a, b: F $(pc = \mathrm{STOP} \vee pc = \mathrm{ERROR})$. Parar em `ERROR` também é terminar.

Usa-se uma **variante** $V(s) = 3\,y + r(pc)$, com $r(\mathrm{LOOP}) = 2$, $r(\mathrm{EVEN}) = r(\mathrm{ODD}) = 1$, $r(\mathrm{STOP}) = r(\mathrm{ERROR}) = 0$, calculada em $n + 2$ bits sem sinal (não há overflow e $V \geq 0$). Prova-se:

- **(a)** o invariante auxiliar
  $J \equiv pc \leq 4 \wedge (pc = \mathrm{ODD} \rightarrow y \text{ ímpar}) \wedge (pc = \mathrm{EVEN} \rightarrow y \text{ par} \neq 0)$, por k-indução;
- **(b)** $J(s) \wedge \neg \mathrm{term}(s) \wedge T(s, s') \rightarrow V(s') < V(s)$;
- **(c)** ausência de bloqueio: todo o estado não terminal que satisfaz J tem sucessor;

- **(d)** como confirmação, um BMC procura um traço de $4n$ transições que evite `STOP` e `ERROR`.

In [ ]:
```
_J, _V = termina_J(M), termina_V(M)
_t0 = time.time()
_ra = kinducao(M, _J, 3)[0]
_rb = "PROVADO" if decresce(M, _J, _V) else "FALSO"
_rc = "PROVADO" if sem_bloqueio(M, _J) else "FALSO"
_tr, _T = unroll(M, 4 * n)
_rd = "PROVADO" if resolve(M.init(_tr[0]), *_T,
                           *[Not(M.terminal(s)) for s in _tr]) is None else
"FALSO"
mo.md(f"""
* (a) invariante auxiliar J: **{_ra}**
* (b) V decresce estritamente fora dos estados terminais: **{_rb}**
* (c) ausência de *deadlock*: **{_rc}**
* (d) BMC: nenhum traço com {4 * n} transições evita `STOP`/`ERROR`: **{_rd}**

({time.time() - _t0:.1f}s). Com $V\\ge0$ bem-fundado, todo o traço termina em no
máximo $V(s_0)$ passos.
""")
```

(a) invariante auxiliar J: **PROVADO com k=1**
(b) V decresce estritamente fora dos estados terminais: **PROVADO**
(c) ausência de *deadlock*: **PROVADO**
(d) BMC: nenhum traço com 32 transições evita `STOP` / `ERROR` : **PROVADO**
(0.6s). Com $V \geq 0$ bem-fundado, todo o traço termina em no máximo $V(s_0)$ passos.

5. Validação exaustiva (sem SMT)

O programa é simulado em Python para **todos** os pares (a, b) com $n = 8$ e a mesma semântica de overflow. Verifica-se que o invariante nunca é violado, quais os pares que terminam em `ERROR` e o máximo de passos.

In [ ]:
```
def simula(a, b, n, com_sinal=False):
    lim = (1 << (n - 1)) if com_sinal else (1 << n)      # valores admissíveis
(não negativos)
    pc, x, y, z = LOOP, a, b, 0
    passos, inv_ok = 0, True
    while pc not in (STOP, ERROR):
        inv_ok &= (x * y + z == a * b)
        if pc == LOOP:
            pc = STOP if y == 0 else (EVEN if y % 2 == 0 else ODD)
        elif pc == EVEN:
            if 2 * x >= lim:
                pc = ERROR
            else:
                x, y, pc = 2 * x, y // 2, LOOP
        else:
            if x + z >= lim:
                pc = ERROR
```

```
            else:
                y, z, pc = y - 1, z + x, LOOP
        passos += 1
    return pc, z, passos, inv_ok


_nn = 8
_linhas = []
for _sinal in (False, True):
    _gama = range(1 << (_nn - 1)) if _sinal else range(1 << _nn)
    _res = [(a, b, *simula(a, b, _nn, _sinal)) for a in _gama for b in _gama]
    _erros = sum(1 for r in _res if r[2] == ERROR)
    _errado = sum(1 for r in _res if r[2] == STOP and r[3] != r[0] * r[1])
    _viol = sum(1 for r in _res if not r[5])
    _peq = sum(1 for r in _res if r[2] == ERROR and r[0] < 2 ** (_nn // 2) and
r[1] < 2 ** (_nn // 2))
    _linhas.append(f"| {'com' if _sinal else 'sem'} sinal | {len(_res)} | {_erros}
| {_errado} | "
                   f"{_viol} | {max(r[4] for r in _res)} | {_peq} |")
mo.md("| n=8 | pares | ERROR | STOP com z≠ab | invariante violado | máx. passos |
ERROR com a,b<2^(n/2) |\n"
      "|---|---|---|---|---|---|---|\n" + "\n".join(_linhas))
```

n=8 pares ERROR STOP com z≠ab invariante violado máx. passos ERROR com a,b<2^(n/2) sem sinal 65536 63568 0 0 31 0 com sinal 16384 15492 0 0 27 32

6. Prova analítica para qualquer n

A k-indução com $n = 16$ é lenta porque o solver faz *bit-blasting* das multiplicações. O argumento matemático, porém, não depende de n. Seja $L = 2^n$ (sem sinal) ou $L = 2^{n-1}$ (com sinal). O overflow é $x + z \geq L$ em `ODD` e $2x \geq L$ em `EVEN`. Enquanto não há overflow, a aritmética módulo 2^n coincide com a dos inteiros.

Hipótese H**:** $a \cdot b < L$ (vale com $a, b < 2^{n/2}$ sem sinal, e com $a, b < 2^{(n-1)/2}$ com sinal).

Invariante
$\text{Inv} \equiv x, y, z \in [0, L) \ \wedge \ x \cdot y + z = a \cdot b \ \wedge \ (pc = \text{ODD} \rightarrow y \text{ ímpar}) \ \wedge \ (pc = \text{EVEN} \rightarrow$
.

É **1-indutivo**:

- `EVEN` : $y \geq 2 \Rightarrow 2x \leq xy \leq xy + z = ab < L$, logo não há overflow; $x'y' = 2x \cdot \frac{y}{2} = xy$;
- `ODD` : $y \geq 1 \Rightarrow x + z \leq xy + z = ab < L$, logo não há *carry*; $x(y-1) + (z + x) = xy + z$;
- `LOOP` , `STOP` : não alteram x, y, z; a paridade vem da guarda.

Logo G $\neg\text{ERROR}$, e em `STOP` ($y = 0$) tem-se $z = ab$. As cláusulas de paridade são o que falta para $k = 1$ (secção 2): sem elas há estados inalcançáveis que quebram o passo.

H **é exata:** se $ab \geq L$, nenhuma execução chega a `STOP` sem overflow, porque $z = ab$ não cabe em n bits. É por isso que, com sinal, $a, b < 2^{n/2}$ já não chega (p.ex. $n = 8$: $15 \cdot 15 = 225 > 127$).

Terminação. $V = 3y + r(pc)$ decresce estritamente em todas as transições não terminais (`LOOP→EVEN/ODD` : $3y + 2 \to 3y + 1$; `EVEN→LOOP` : $\to 3\frac{y}{2} + 2 \leq 3y - 1$; `ODD→LOOP` : $\to 3y - 1$; `LOOP→STOP` e erros $\to 0$), e $V \geq 0$ é um inteiro, logo o programa termina para qualquer n. Há no máximo $n - 1$ passos `EVEN` e n passos `ODD` , portanto no máximo $4n - 1$ transições.

7. Conclusões

alínea	resultado
1. SFOTS	variáveis pc, a, b, x, y, z; estado inicial $pc = \mathrm{LOOP}$, $x = a$, $y = b$, $z = 0$; 9 ligações com precedência das de erro; estado de erro $pc = \mathrm{ERROR}$
2. $xy + z = ab$	invariante provado por k-indução (secção 2); $k = 1$ falha por estados inalcançáveis
3. segurança com $a, b < 2^{n/2}$	provada, diretamente ou reforçando com o invariante; sem a condição o erro é alcançável
4. terminação	variante $V = 3y + r(pc)$ com invariante auxiliar e ausência de bloqueio (secção 4)

Os valores de k e os resultados `PROVADO` / `FALSO` das secções 2 a 4 são calculados pelo solver para o modo (com ou sem sinal) escolhido no topo, com $n = 8$.

8. Impacto de pequenas modificações nos requisitos

| modificação | alteração | efeito | |---|---|---| | overflow **com sinal** | ligar o interruptor "com sinal" | a condição $a, b < 2^{n/2}$ pode deixar de garantir segurança, porque o produto pode atingir 2^{n-1}, que já muda o sinal. A extensão exata passa a usar `BVSExt` | | condição mais fraca $a \cdot b < 2^n$ | `pequenos` com `BVULT(BVMul(W(a), W(b)), 2**n)` | o argumento do reforço só usa $ab < 2^n$ | | geral em n | com $ab < L$, o invariante $xy + z = ab$ com paridade é 1-indutivo, logo não há `ERROR` ; $V = 3y + r(pc)$ garante a terminação em $\leq 4n - 1$ passos (secção 6) | | outra ordem de teste | `guards` e `trans` | o invariante e a variante mantêm-se; com sinal, `y > 0` é necessário para a terminação | | remover a precedência do erro | tirar `Not(ovf…)` das guardas normais | aparecem traços que continuam após o overflow com valores truncados |

9. Testes

Os testes estão fora do notebook, em `test_mult.py`, e importam daqui a classe `Mult` e as funções de verificação (declaradas com `@app.class_definition` e `@app.function`):

```
pip install pytest
pytest test_mult.py -v
```

10. Utilização de LLMs

Declaração das ferramentas de IA usadas na realização deste trabalho, com as respetivas conversas:

ferramenta	utilização	ligação
Google AI Mode	pesquisa de apoio	conversa
Claude	melhor interpretação do enunciado	conversa
Claude	estrutura de pastas e ficheiros do projeto (*file tree*)	conversa